

OS LAB-4
DATE: 16/3/26

SUMANT SHRIDHAR
USN: 1BM24CS299
BATCH: 4K2

1) Round Robin:

```
#include <stdio.h>

struct Process {
    int pid, at, bt, rt, ct, tat, wt;
};

int main() {
    int n, tq, i, time = 0, completed = 0;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &tq);

    struct Process p[n];

    printf("Enter arrival time and burst time:\n");
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("P%d AT: ", i + 1); scanf("%d", &p[i].at);
        printf("P%d BT: ", i + 1); scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
        p[i].ct = 0;
    }

    int queue[100], front = 0, rear = 0;
    int inQueue[n];
    for (i = 0; i < n; i++) inQueue[i] = 0;

    // Enqueue processes that arrive at time 0
    for (i = 0; i < n; i++)
        if (p[i].at <= time) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }

    while (completed < n) {
        // If queue empty, advance time
        if (front == rear) {
            time++;
            for (i = 0; i < n; i++)
                if (!inQueue[i] && p[i].at <= time) {
                    queue[rear++] = i;
                }
        }
    }
}
```

```

        inQueue[i] = 1;
    }
    continue;
}

int idx = queue[front++];

if (p[idx].rt > tq) {
    time += tq;
    p[idx].rt -= tq;

    // Enqueue newly arrived processes
    for (i = 0; i < n; i++)
        if (!inQueue[i] && p[i].at <= time) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }

    queue[rear++] = idx; // Re-enqueue at back

} else {
    time += p[idx].rt;
    p[idx].rt = 0;
    p[idx].ct = time;
    completed++;

    // Enqueue newly arrived processes
    for (i = 0; i < n; i++)
        if (!inQueue[i] && p[i].at <= time) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }
}
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

printf("\nAverage Turnaround Time: %.2f", avg_tat / n);

```

```
printf("\nAverage Waiting Time : %.2f\n", avg_wt / n);

return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter time quantum: 2
Enter arrival time and burst time:
P1 AT: 0
P1 BT: 3
P2 AT: 2
P2 BT: 2
P3 AT: 3
P3 BT: 5
P4 AT: 4
P4 BT: 4
P5 AT: 6
P5 BT: 1

PID    AT    BT    CT    TAT    WT
P1      0     3     5     5     2
P2      2     2     4     2     0
P3      3     5    15    12     7
P4      4     4    14    10     6
P5      6     1    10     4     3

Average Turnaround Time: 6.60
Average Waiting Time : 3.60
```


2) Multi Level Scheduling

```
#include <stdio.h>

struct Process {
    int pid, at, bt, type, ct, tat, wt;
};

int main() {

    int n, i, j, time = 0;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    printf("Enter arrival time, burst time, and type (0 = System, 1 = User):\n");

    for (i = 0; i < n; i++) {

        p[i].pid = i + 1;

        printf("P%d AT: ", i + 1);
        scanf("%d", &p[i].at);

        printf("P%d BT: ", i + 1);
        scanf("%d", &p[i].bt);

        printf("P%d Type (0=System,1=User): ", i + 1);
        scanf("%d", &p[i].type);
    }

    /* Sort processes:
       1. System processes first
       2. Within same type -> sort by arrival time
    */

    for (i = 0; i < n - 1; i++) {

        for (j = 0; j < n - i - 1; j++) {

            int swap = 0;

            if (p[j].type > p[j + 1].type)
```

```

        swap = 1;

    else if (p[j].type == p[j + 1].type &&
             p[j].at > p[j + 1].at)
        swap = 1;

    if (swap) {
        struct Process temp = p[j];
        p[j] = p[j + 1];
        p[j + 1] = temp;
    }
}

/* Execute processes */

for (i = 0; i < n; i++) {

    if (time < p[i].at)
        time = p[i].at;

    time += p[i].bt;

    p[i].ct = time;
}

printf("\nPID\tAT\tBT\tType\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++) {

    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;

    avg_tat += p[i].tat;
    avg_wt += p[i].wt;

    printf("P%d\t%d\t%d\t%s\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].at,
           p[i].bt,
           (p[i].type == 0 ? "Sys" : "User"),
           p[i].ct,
           p[i].tat,
           p[i].wt);
}

```

```

printf("\nAverage Turnaround Time: %.2f", avg_tat / n);
printf("\nAverage Waiting Time : %.2f\n", avg_wt / n);

return 0;
}

```

OUTPUT:

```

Enter number of processes: 5
Enter arrival time, burst time, and type (0 = System, 1 = User):
P1 AT: 0
P1 BT: 3
P1 Type (0=System,1=User): 1
P2 AT: 2
P2 BT: 2
P2 Type (0=System,1=User): 0
P3 AT: 3
P3 BT: 5
P3 Type (0=System,1=User): 1
P4 AT: 4
P4 BT: 4
P4 Type (0=System,1=User): 0
P5 AT: 6
P5 BT: 1
P5 Type (0=System,1=User): 1

PID    AT    BT    Type    CT    TAT    WT
P2      2     2     Sys     4     2     0
P4      4     4     Sys     8     4     0
P1      0     3     User    11    11     8
P3      3     5     User    16    13     8
P5      6     1     User    17    11    10

Average Turnaround Time: 8.20
Average Waiting Time : 5.20

```


16/3/26

1)

Round Robin Scheduling

#include <stdio.h>

struct Process {

int pid, at, bt, rt, ct, tat, wt;

};

int main() {

int n, tq, i, time = 0, completed = 0;
float avg_tat = 0, avg_wt = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

printf("Enter time quantum: ");

scanf("%d", &tq);

struct Process p[n];

printf("Enter AT and BT: \n");

for(i = 0; i < n; i++) {

p[i].pid = i + 1;

pf("P: %d AT: ", i + 1); if("%d", &p[i].at);

pf("P: %d BT: ", i + 1); if("%d", &p[i].bt);

p[i].rt = p[i].bt;

p[i].ct = 0;

}

int queue[100], front = 0, rear = 0;

int inQueue[n];

for(i = 0; i < n; i++) inQueue[i] = 0;

// Enqueue processes that arrive at t = 0

2

```
for (i = 0; i < n; i++)
    if (p[i].at <= time) {
        queue[rear++] = i;
        inQueue[i] = 1;
    }
```

```
while (completed < n) {
    if (front == rear) { // Queue Empty
        time++;
        for (i = 0; i < n; i++)
            if (!inQueue[i] && p[i].at <= time) {
                queue[rear++] = i;
                inQueue[i] = 1;
            }
        continue;
    }
```

```
    int idx = queue[front++];
    if (p[idx].rt > tq) {
        time += tq;
        p[idx].rt -= tq;
    }
```

```
    for (i = 0; i < n; i++)
        if (!inQueue[i] && p[i].at <= time) {
            queue[rear++] = i;
            inQueue[i] = 1;
        }
```

```
    queue[rear++] = idx;
```

```
    else {
        time += p[idx].rt;
        p[idx].rt = 0;
        p[idx].ct = time;
        completed++;
    }
```



```

for(i = 0; i < n; i++) {
    if (!inQueue[i] && p[i].at <= time) {
        queue[rear++] = i;
        inQueue[i] = 1;
    }
}

```

```

printf("In PID \t AT \t BT \t CT \t TAT \t WT \n");
for(i = 0; i < n; i++) {

```

```

    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg-tat += p[i].tat;
    avg-wt += p[i].wt;
}

```

```

printf("P: %.d \t T: %.d \t B: %.d \t C: %.d \t T: %.d \n",
       p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}

```

```

printf("In Average TAT: %.2f", avg-tat/n);
printf("In Average WT: %.2f", avg-wt/n);

```

~~return 0;~~

```

printf("In Gantt Chart: \n");
for(i = front; i < rear; i++) {
    printf("%d -> ", queue[i]);
}

```

```

printf("In ANULL");

```

```

return 0;

```


2

Output:

Enter no of processes : 5

Enter TSD : 2

Enter AT and BT:

P1 AT : 0

P1 BT : 3

P2 AT : 2

P2 BT : 2

P3 AT : 3

P3 BT : 5

P4 AT : 4

P4 BT : 4

P5 AT : 6

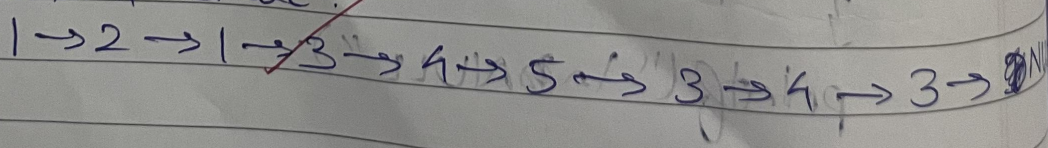
P5 BT : 1

PID	AT	BT	CT	TAT	WT
P1	0	3	5	5	2
P2	2	2	4	2	0
P3	3	5	15	12	7
P4	4	4	14	10	6
P5	6	1	10	4	3

Average TAT : 6.60

Average WT : 3.60

Gantt Chart:



2) Multi-Level Queue Scheduling

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid, at, bt, type, ct, tot, wt;
};
```

```
int main() {
```

```
    int n, i, j, time = 0;
```

```
    float avg_tot = 0, avg_wt = 0;
```

```
    printf("Enter no of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    printf("Enter AT, BT, and type
```

```
    (0 - System 1 - User): \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("P%d AT: ", i + 1); scanf("%d", &p[i].at);
```

```
        printf("P%d BT: ", i + 1); scanf("%d", &p[i].bt);
```

```
        printf("P%d Type: ", i + 1); scanf("%d", &p[i].type);
```

```
    }
```

// Sort by type, then by AT within that

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = 0; j < n - 1; j++) {
```

```
            int swap = 0;
```

```
            if (p[j].type > p[j + 1].type)
```

```
                swap = 1;
```


2

```

else if (p[j].type == p[j+1].type &&
p[j].at > p[j+1].at)
    swap = 1;

```

```

if (swap) {
    struct Process temp = p[j];
    p[j] = p[j+1];
    p[j+1] = temp;
}
}
}

```

```

for (i = 0; i < n; i++) {
    if (time < p[i].at) time = p[i].at;
    time += p[i].bt;
    p[i].ct = time;
}

```

```

printf("\n PID \t AT \t BT \t CT \t TAT \t WT \n");

```

```

for (i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}

```

```

printf("P \t d \t t \t c \t a \t b \t w \n");
printf("P \t d \t t \t c \t a \t b \t w \n");
p[i].pid, p[i].at, p[i].bt,
p[i].type == 0 ? "Sys" : "User",
p[i].ct, p[i].tat, p[i].wt

```

```

printf("In Average TAT: %.2f", avg_tat/n);
printf("In Average WT: %.2f", avg_wt/n);
return 0;

```

O/P:
~~Output:~~

Enter A
P1 AT
P1 BT
P1 Type
P2 AT
P2 BT
P2 Tc
P3 AT
P3 BT
P3 Tc
P4 AT
P4 BT
P4 Tc
P5 AT
P5 BT
P5 Tc

PID
P2
P4
P1
P3
P5

Average
Average

O/P:

Q. Enter no of processes: 5

Enter AT, BT, Type (0 - System, 1 - User):

P1 AT: 0

P1 BT: 3

P1 Type: 1

P2 AT: 2

P2 BT: 2

P2 Type: 0

P3 AT: 3

P3 BT: 5

P3 Type: 1

P4 AT: 4

P4 BT: 4

P4 Type: 0

P5 AT: 6

P5 BT: 1

P5 Type: 1

PID	AT	BT	Type	CT	TAT	WT
P2	2	2	Sys	4	2	0
P4	4	4	Sys	8	4	0
P1	0	3	User	11	11	8
P3	3	5	User	16	13	8
P5	6	1	User	17	11	10

Average TAT: 8.20

Average WT: 5.20

SPL